

Pipelined Reliable Data Transfer Protocols

(Extension of rdt3.0 – Overcoming the Stop-and-Wait Limitation)

1 Introduction

The protocol **rdt3.0** provides reliable data transfer over unreliable channels using acknowledgments (ACKs) and retransmissions. However, it is a **stop-and-wait protocol**, meaning the sender must wait for an ACK before sending the next packet. While this ensures reliability, it results in **very poor performance** in modern high-speed networks.

Problem

In stop-and-wait protocols, the sender spends most of the time waiting instead of transmitting data. Even with high bandwidth, the effective throughput is drastically reduced due to long propagation delays.

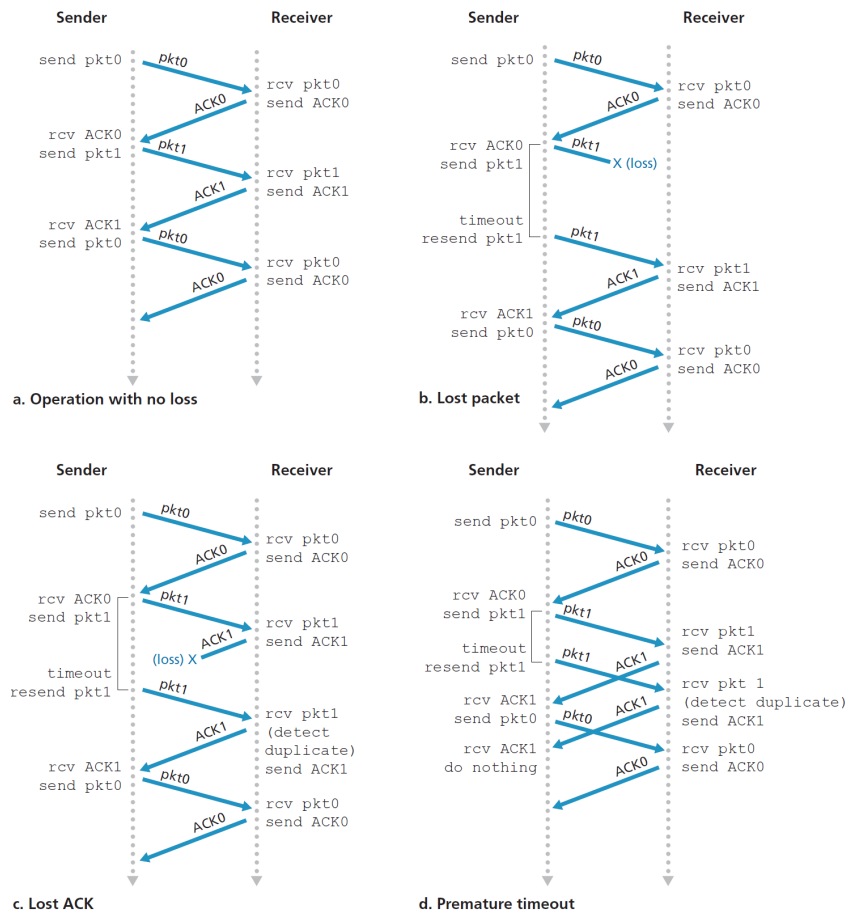


Figure 1: **Operation of rdt3.0:** The alternating-bit protocol illustrating stop-and-wait behavior.

2 Performance Analysis of Stop-and-Wait Protocol

2.1 Scenario Setup

- Two hosts: one on the West Coast and one on the East Coast of the United States.
- Round-trip propagation delay (**RTT**) = 30 milliseconds.
- Channel rate (**R**) = 1 Gbps = 10^9 bits/sec.
- Packet size (**L**) = 1,000 bytes = 8,000 bits.

2.2 Transmission Time Calculation

$$d_{\text{trans}} = \frac{L}{R} = \frac{8000 \text{ bits}}{10^9 \text{ bits/sec}} = 8 \mu s$$

Thus, the time to transmit one packet onto the link is only $8 \mu s$.

2.3 Timeline of Stop-and-Wait Operation

1. The sender starts sending at $t = 0$.
2. The last bit of the packet is transmitted at $t = L/R = 8 \mu s$.
3. The packet travels across the country (one-way delay ≈ 15 ms).
4. Receiver sends an ACK immediately after receiving the last bit.
5. ACK reaches sender at $t = RTT + L/R = 30.008$ ms.
6. Only then can the sender send the next packet.

Observation

During 30.008 ms, the sender was busy transmitting for only 0.008 ms!

2.4 Sender Utilization

$$U_{\text{sender}} = \frac{L/R}{RTT + L/R} = \frac{0.008}{30.008} = 0.00027$$

That is, the sender is busy only **0.027% of the time**.

2.5 Effective Throughput

$$\text{Throughput} = U_{\text{sender}} \times R = 0.00027 \times 10^9 = 267 \text{ kbps}$$

Even with a **1 Gbps link**, the actual data throughput is only about **267 kbps** — an extreme underutilization!

Conclusion

Stop-and-wait protocols severely waste network capacity when the product of bandwidth and delay is large.

3 The Solution: Pipelining

3.1 Concept

Instead of waiting for each ACK before sending the next packet, the sender transmits multiple packets consecutively before receiving acknowledgments. This technique is called **pipelining**.

Analogy

Analogy: Imagine a factory conveyor belt (the network link). In stop-and-wait, only one box (packet) is placed on the belt at a time. In pipelining, multiple boxes are placed sequentially, keeping the belt fully utilized.

3.2 Effect of Pipelining

If the sender can send, for example, three packets before waiting for ACKs:

$$U_{\text{sender}}^{(3)} = 3 \times 0.00027 = 0.00081$$

Utilization and throughput increase proportionally with the pipeline depth.

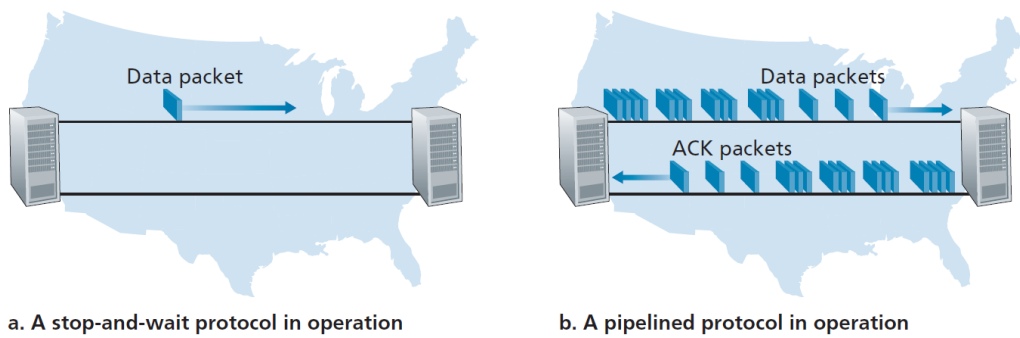


Figure 2: **Stop-and-Wait vs. Pipelined Protocol:** Pipelining allows multiple packets in transit simultaneously.

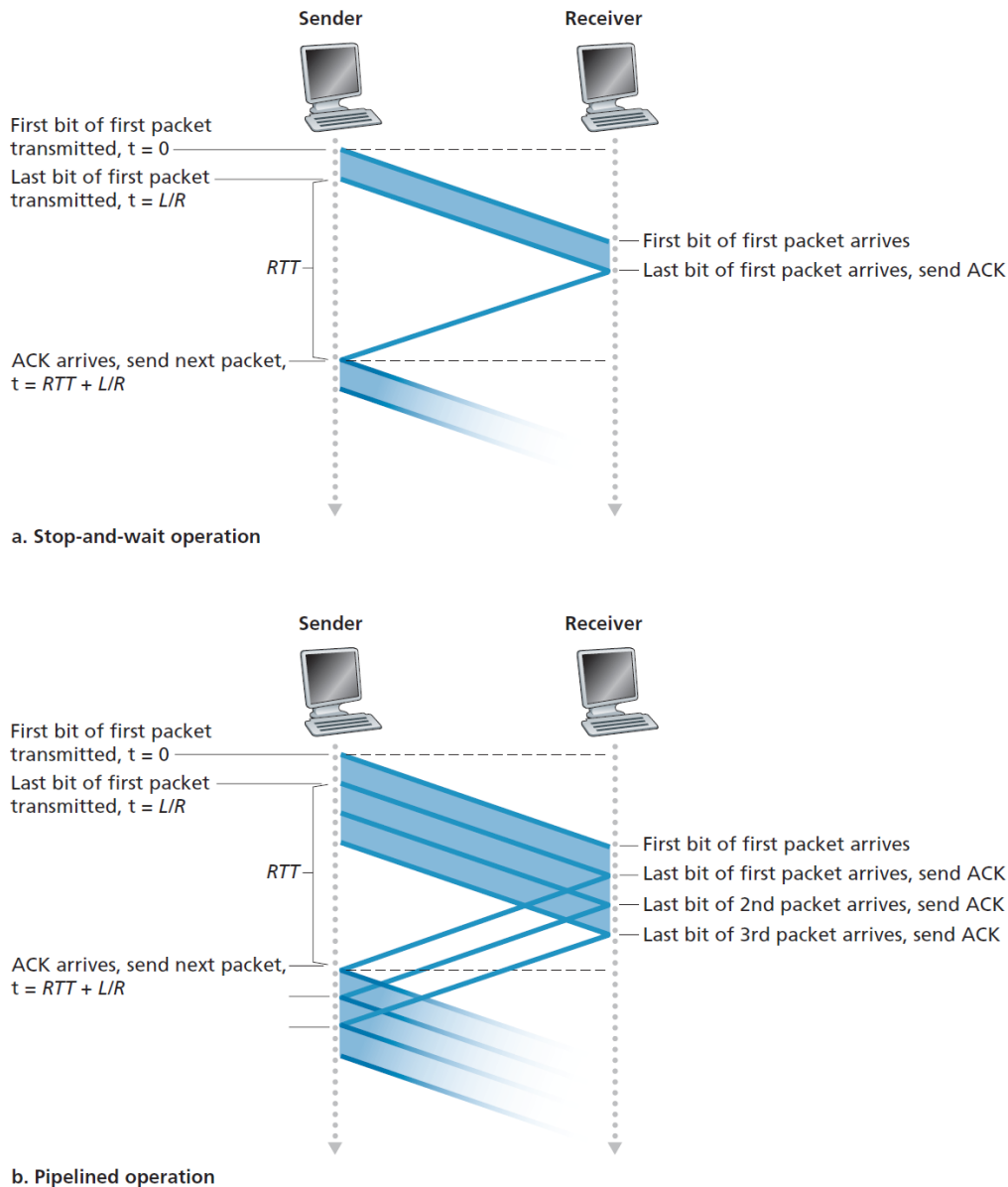


Figure 3: **Stop-and-Wait and Pipelined Sending:** Timing diagram comparison of both operations.

3.3 Consequences of Pipelining

To implement pipelining effectively, reliable data transfer protocols must address the following:

1. **Increased Sequence Number Range:** Each in-transit packet must have a unique sequence number since multiple packets may be unacknowledged simultaneously.
2. **Buffering Requirements:**
 - The sender must buffer all transmitted but unacknowledged packets (for possible retransmission).
 - The receiver may need to buffer out-of-order packets (depending on the protocol type).

3. Error Recovery Mechanisms: Two main approaches exist for handling lost or corrupted packets in pipelined systems:

- **Go-Back-N (GBN):** Retransmit all unacknowledged packets upon detecting a loss.
- **Selective Repeat (SR):** Retransmit only the specific lost or corrupted packets.

3.4 Advantages

- Drastically improves utilization of the network link.
- Increases throughput proportionally to the number of outstanding packets.
- Makes efficient use of high-speed links and long-delay channels.

4 Summary

Comparison Summary

Stop-and-Wait:

- Reliable but inefficient for high bandwidth-delay product links.
- Sender utilization extremely low (0.027% in example).

Pipelined Protocols:

- Allow multiple packets in transit simultaneously.
- Improve channel utilization and throughput.
- Require larger sequence number space and buffering.
- Lead to advanced protocols like Go-Back-N and Selective Repeat.

In essence: Pipelining transforms reliable data transfer protocols from idle stop-and-wait systems into highly efficient continuous-flow systems — a crucial advancement for high-speed modern networks.